

When organizations store raw data in **cloud storage systems like Amazon S3**, Snowflake needs a **secure mechanism** to access those files without exposing AWS credentials.

Snowflake solves this using **Storage Integration with AWS IAM Roles**.

This architecture allows Snowflake to **securely access S3 buckets for loading and unloading data** while maintaining strong security controls.

In this post, I am explaining the **complete end-to-end architecture and setup of Snowflake S3 Storage Integration**.

What is Storage Integration in Snowflake?

A **Storage Integration** is a Snowflake object that securely connects Snowflake with **external cloud storage services** such as:

- Amazon S3
- Azure Blob Storage
- Google Cloud Storage

Instead of manually providing **AWS Access Keys**, Snowflake uses **IAM roles and trust relationships** for secure authentication.

Benefits of Storage Integration

- ✓ No need to store AWS credentials inside Snowflake
 - ✓ Secure cross-account access
 - ✓ Centralized access management
 - ✓ Supports multiple stages and buckets
 - ✓ Recommended best practice for production systems
-

Integration Architecture Overview

The Snowflake–AWS integration follows a **secure authentication flow**.

Step-by-step architecture:

- 1 Snowflake creates a **Storage Integration object**
- 2 Snowflake automatically generates an **AWS IAM User**
- 3 AWS administrators create an **IAM Policy**
- 4 The policy is attached to an **IAM Role**
- 5 A **trust relationship** is created between the IAM Role and the Snowflake IAM User
- 6 Snowflake users create **External Stages** referencing the Storage Integration

7 Snowflake verifies the IAM permissions before loading or unloading data

This ensures **secure communication between Snowflake and AWS S3**.

AWS Access Control Requirements

For Snowflake to access data stored in S3, specific **S3 permissions must be granted**.

Required Permissions

Permission	Purpose
s3:GetBucketLocation	Determines bucket region
s3:GetObject	Reads objects in the bucket
s3:GetObjectVersion	Access object versions
s3:ListBucket	Lists files inside bucket

These permissions allow Snowflake to **read files from S3 buckets**.

Additional Permissions for Advanced Operations

Some Snowflake SQL operations require additional permissions.

Permission	SQL Action
s3:PutObject	Unload Snowflake data to S3
s3:DeleteObject	Remove staged files
s3:DeleteObjectVersion	Remove versioned files

These permissions allow Snowflake to **write and manage objects in S3**.

Step 1 – Create IAM Policy in AWS

The first step is creating an **IAM policy** that defines the permissions Snowflake will have on the S3 bucket.

An IAM policy is a **JSON document that specifies access rules**.

Steps

- 1** Login to **AWS Management Console**
- 2** Navigate to **IAM (Identity and Access Management)**
- 3** Select **Policies**
- 4** Click **Create Policy**

5 Choose **JSON** tab

Example policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:PutObject",
        "s3:GetObject",
        "s3:GetObjectVersion",
        "s3:DeleteObject",
        "s3:DeleteObjectVersion"
      ],
      "Resource": "arn:aws:s3:::skillscaleraw/*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "s3:ListBucket",
        "s3:GetBucketLocation"
      ],
      "Resource": "arn:aws:s3:::skillscaleraw"
    }
  ]
}
```

Explanation

The first section allows operations on **objects inside the bucket**.

The second section allows operations on **the bucket itself**.

After creating the policy, we attach it to an IAM role.

Step 2 – Create IAM Role in AWS

Next, we create an **IAM Role** that Snowflake will assume when accessing the S3 bucket.

Steps

- 1 Open **AWS IAM Console**
- 2 Select **Roles**
- 3 Click **Create Role**
- 4 Choose **Another AWS Account**
- 5 Enter your **AWS Account ID temporarily**

6 Enable **External ID**

7 Attach the **IAM policy created earlier**

8 Create the role

Once the role is created, copy the **Role ARN**.

Example:

arn:aws:iam::001234567890:role/myrole

This ARN will be used when creating the **Snowflake Storage Integration**.

Step 3 – Create Storage Integration in Snowflake

Now we create the **Storage Integration object in Snowflake**.

This object stores:

- AWS IAM Role
- Allowed storage locations
- Authentication configuration

Example SQL:

```
CREATE STORAGE INTEGRATION s3_int
TYPE = EXTERNAL_STAGE
STORAGE_PROVIDER = S3
ENABLED = TRUE
STORAGE_AWS_ROLE_ARN = 'arn:aws:iam::001234567890:role/myrole'
STORAGE_ALLOWED_LOCATIONS = ('s3://mybucket1/mypath1/');
```

Explanation of Parameters

Parameter	Description
TYPE	Defines integration type
STORAGE_PROVIDER	Specifies S3
ENABLED	Enables integration
STORAGE_AWS_ROLE_ARN	IAM role used for authentication
STORAGE_ALLOWED_LOCATIONS	Allowed S3 paths

This configuration ensures Snowflake can **only access specific S3 locations**.

Step 4 – Retrieve Snowflake IAM User

After creating the integration, we run:

```
DESC INTEGRATION s3_int;
```

This command returns important values.

Example output fields:

- STORAGE_AWS_IAM_USER_ARN
- STORAGE_AWS_ROLE_ARN
- STORAGE_AWS_EXTERNAL_ID

STORAGE_AWS_IAM_USER_ARN

This is the **IAM user created automatically by Snowflake**.

Important point:

Snowflake creates **only one IAM user per Snowflake account**.

All storage integrations use this IAM user.

STORAGE_AWS_EXTERNAL_ID

The external ID is used to establish a **secure trust relationship between Snowflake and AWS**.

It prevents unauthorized access from other AWS accounts.

Example format:

```
snowflake_account_SFRole=snowflake_role_id_random_id
```

Step 5 – Configure Trust Relationship in AWS

Now we update the **IAM Role trust policy** so Snowflake can assume the role.

Example trust policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "<snowflake_user_arn>"
      },
      "Action": "sts:AssumeRole",
      "Condition": {
        "StringEquals": {
          "sts:ExternalId": "<snowflake_external_id>"
        }
      }
    }
  ]
}
```

```
}  
}  
]  
}
```

Explanation

- **Principal** identifies the Snowflake IAM user
- **sts:AssumeRole** allows Snowflake to use the IAM role
- **External ID** ensures secure cross-account authentication

This step completes the **secure connection between Snowflake and AWS S3**.

Step 6 – Create External Stage in Snowflake

Finally, we create an **external stage that references the storage integration**.

Example:

```
CREATE STAGE my_s3_stage  
STORAGE_INTEGRATION = s3_int  
URL = 's3://bucket1/path1'  
FILE_FORMAT = my_csv_format;
```

Required Privileges

```
GRANT CREATE STAGE ON SCHEMA public TO ROLE myrole;  
GRANT USAGE ON INTEGRATION s3_int TO ROLE myrole;
```

The stage now acts as a **pointer to the S3 bucket location**.

Snowflake can now **load or unload data using COPY commands**.

Complete Data Flow Architecture

```
graph TD  
    A[AWS S3 Bucket] --> B[IAM Policy]  
    B --> C[IAM Role]  
    C --> D[Snowflake Storage Integration]  
    D --> E[Trust Relationship]  
    E --> F[External Stage]  
    F --> G[COPY INTO Command]  
    G --> H[Snowflake Tables]
```

Key Takeaways

- ✓ Storage Integration provides secure authentication for cloud storage
- ✓ Snowflake automatically creates an IAM user for integration
- ✓ IAM policies define S3 access permissions
- ✓ IAM roles enable Snowflake to assume AWS permissions
- ✓ Trust relationships ensure secure cross-account access
- ✓ External stages allow Snowflake to load data directly from S3

ROHITH KUMAR